

Remaining Useful Life Prediction Considering Data and Model Uncertainties

Yuling Zhan^{1,2,*}, Ziqi Wang^{1,2}, and Zhengguo Xu^{1,2}

¹College of Control Science and Engineering, Zhejiang University, Hangzhou, China

²Huzhou Institute, Zhejiang University, Huzhou, China

sarajlbb@zju.edu.cn, wangziqincepu@163.com, xzg@zju.edu.cn

*corresponding author

Abstract—Remaining useful life (RUL) prediction plays a great part in prognostics health management (PHM). Recently, machine learning (ML) methods have been widely used in RUL prediction due to their scalability and generalization. However, most of the research focuses on RUL point estimation and neglects the uncertainties that are caused by data noise and modeling inaccuracy. To get more reliable and practical results, an RUL interval prediction method with data and model uncertainty quantification is proposed. Firstly, a series of neural network models with various parameters and structures are constructed and they are trained with bootstrapped data. After that, the outputs of the models are fused to form new time series labels which contain data and model uncertainties, and a new training set consisting of sequential features and new labels is generated. Finally, a heteroscedastic neural network (HNN) is trained to capture both uncertainties as well as output RUL point estimation. The effectiveness of the proposed method is validated on CMAPSS datasets.

Keywords—remaining useful life; interval prediction; uncertainty quantification; bootstrap

1. INTRODUCTION

It is vital to assuring reliability, safety, and security in industrial systems. Prognostics and health management (PHM) was born to provide fault prediction with condition-based monitoring (CBM) technology and gives preventive maintenance strategies to cut down repair costs. Remaining useful life (RUL) estimation is a key part of PHM which gets earlier fault information and gives timely failure alerts. Since RUL prediction is able to mitigate insufficient maintenance and excessive maintenance, it is getting more and more attention in research.

Generally, the RUL prediction methods can be classified into two categories: the mechanism-based methods and the data-driven methods [1]. Mechanism-based methods construct degradation models according to physical principles. Paris-Erdogan model [2] proposed by Paris is a classic one that focuses on the fatigue crack expansion problem. Although this type of approach can provide accurate predictions, they are usually limited by model complexity and poor generality. Data-driven method includes the statistical approach and the machine learning (ML) approach [3]. The former includes auto-regressive integrated moving average model (ARIMA) [4], stochastic process-based methods [5][6], and Bayesian filtering methods [7]. These approaches are

usually constrained to specific statistical assumptions, however, the distribution of samples is not known most of the time in reality.

ML approach directly uses data to predict RUL, compared with other methods, it is more general with fewer limitations [8]. Existing researches mostly focus on RUL point estimation which aims to predict the time to failure accurately. However, the predictive process is full of uncertainties which may be caused by sensor quality, measurement technology, the veracity of the model, etc. [9] Decision-making without uncertainty considering would be overconfident or over-conservative, and accordingly lead to maintenance not in time or additional economic losses. Therefore, uncertainties matter in maintenance decisions and can not be ignored. Some research has paid attention to the uncertainty quantification problem. Owing to the capability of uncertainty representation, stochastic process approaches are often adopted to predict RUL [10][11][12]. However, degradation functions have to be constructed first which is challenging for complex systems and high-dimensional situations.

ML approaches can solve the above problem and learn degradation information from training data and output prediction directly. Bayesian neural network (BNN) is the most applied in ML approaches to consider uncertainty estimation, it regards weights in the network as distributions and outputs prediction distribution as well [13][14][15]. The main drawback of BNN is the high computational complexity which is caused by numerous parameters in posterior inference. Blundell et al. [16] proposed variational inference to fit posterior distribution approximately. It mitigates the computational burden but is still time costly. A novel method came up with Gal [17] to approximate Bayesian inference with Monte Carlo dropout and simplify the calculation significantly. Peng et al. [18] applied the approach in [17] to RUL prediction based on CNN and LSTM networks, although the realization is more convenient, the accuracy is inferior to before. In addition to BNN based method, another main category is heteroscedastic neural network (HNN) [19][20][21] which outputs mean and variance simultaneously. This kind of method assumes the RUL result as a distribution and uses maximization likelihood estimation (MLE) to train the network. This approach is easy and efficient but it only captures the data uncertainty.

Since uncertainties in RUL prediction may originate from data and models, a comprehensive estimation is needed to provide a better prediction. A novel method that combines

bootstrap and HNN to realize RUL point estimation and uncertainty quantification is proposed. The main contributions of this paper are as follows.

1) The proposed method considers two aspects of uncertainties in RUL prediction. The first part is data uncertainty caused by data measurement and acquisition, and the bootstrap method is utilized to capture it. Another part is model uncertainty originates from modeling inaccuracy, and it is expressed by different models with various structures and parameters. A series of neural network models are constructed and trained with the above operations, and the results are fused to constitute the new RUL labels which contain two types of uncertainties.

2) An HNN model is leveraged to quantify both uncertainties in the proposed method. A CNN-LSTM-based HNN model is constructed to output RUL point estimation and uncertainty value simultaneously through MLE and it is trained with the newly generated training datasets.

3) The performance of the raised method exceeds other general approaches both on point estimation and interval prediction. The framework of this method is generic for other applications of RUL prediction.

The rest of the paper is organized as follows. In section 2, the preliminaries of Bootstrap and HNN are introduced. In section 3, the architecture of the proposed method and the adopted evaluation metrics are provided. The effectiveness of the proposed method is validated on CMAPSS datasets described in section 4, and section 5 presents the conclusions.

2. PRELIMINARIES

2.1. Bootstrap

Bootstrap [22] is one of the popular ensemble methods. Compared to another classic ensemble study, namely boosting [23], it has the advantage of parallel and efficient computing. Bootstrap induces diversity and randomness into prediction and can improve predictive accuracy. Because of the randomness property, bootstrap has the capacity to capture uncertainty during the training process. It is easy to operate and is independent of data distribution.

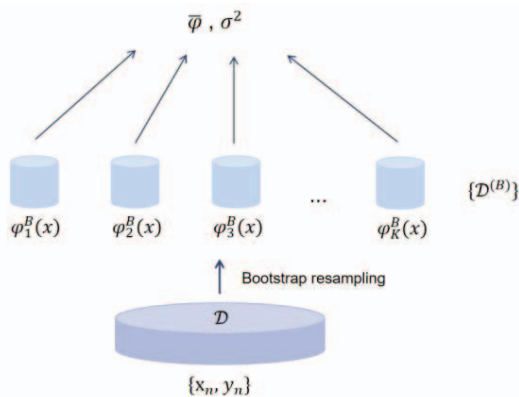


Figure 1. Bootstrap method

Let $\mathcal{D}$ denote a learning set that includes $\{(x_n, y_n), n =$

$1, 2, \dots, N\}$, where x_n is training data and y_n is regression label. $\varphi(x, \mathcal{D})$ is the trained predictor with input x and output y . Bootstrap means resampling part of data from the learning set with replacing and constituting a group of new datasets $\{\mathcal{D}^{(B)}\}$. Each $\mathcal{D}^{(B)}$ contains B samples from $\mathcal{D}$, drawn with randomness. Accordingly, a set of a new predictor $\varphi^B(x)$ is formed by $\{\varphi(x, \mathcal{D}^{(B)})\}$. Figure 1 specifically illustrates the process of the bootstrap. Changes in $\mathcal{D}$ will influence the predictor, the smaller change, φ^B get closer to φ .

Each predictor in the new set produces an output, after that, an ensemble method is used to generate the final mean and variance as follows:

$$\bar{\varphi} = \frac{1}{K} \sum_{k=1}^K \varphi^B(x) \quad (1)$$

$$\sigma^2 = \frac{1}{K-1} \sum_{k=1}^K (\varphi^B(x) - \bar{\varphi})^2 \quad (2)$$

Where K is the number of total predictors, $\bar{\varphi}$ and σ^2 denote predictive mean and variance respectively.

2.2. Heteroscedastic Neural Network

RUL prediction is a time-series regression problem. Input $X \in \mathbb{R}^{t \times n}$, X is a sequential feature vector, t and n represent the time length and the number of features respectively. Output $Y \in \mathbb{R}^t$, Y is the predictive result.

Although it is a point prediction with only one output at each time step, y is assumed to be an uncertain value and the uncertainty can be described by variance σ^2 . Both y and σ^2 are input-dependent. A heteroscedastic neural network (HNN) [24] is consequently constructed to output prediction and uncertainty simultaneously as depicted in Figure 2.

Let Y be conditional on X and assume it is normal distribution $P(Y|X) \sim \mathcal{N}(\mu(X), \sigma^2(X))$. $\mu(X)$ and $\sigma^2(X)$ are two outputs of the HNN. MLE is adopted to search for parameters, and negative log-likelihood (NLL) is chosen to optimize the network. Loss function can be concluded as:

$$Loss = \frac{1}{2} \log(\sigma^2(X)) + \frac{(Y - \mu(X))^2}{2\sigma^2(X)} + constant \quad (3)$$

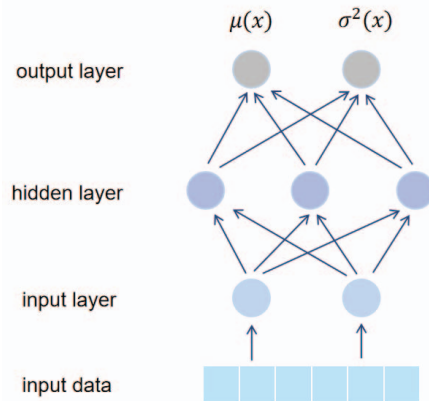


Figure 2. Heteroscedastic neural network (HNN)

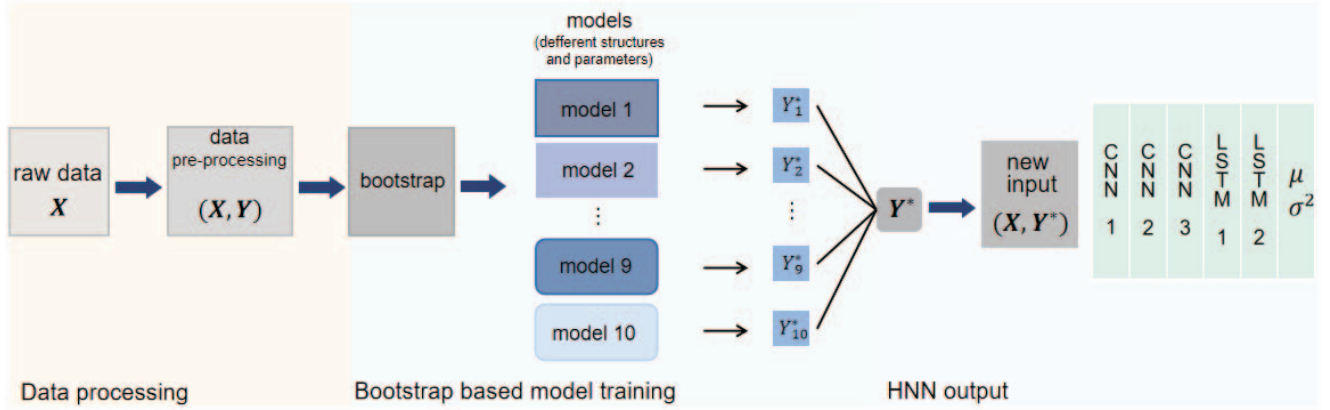


Figure 3. The framework of the proposed method

where $\sigma^2(X)$ is positive, it can be constrained for instance with the sigmoid function.

3. METHODOLOGY

This section introduces a novel method to capture uncertainty during RUL prediction.

3.1. Architecture

Influenced by sensor noise, measure errors, inaccuracy of models, etc, the RUL prediction is always full of uncertainty. We classify them into data uncertainty and model uncertainty. The prediction architecture mainly consists of two parts, that is the bootstrap-based model training step and the HNN output step. Figure 3 depicts the whole framework of the proposed Bootstrap-HNN model.

1) Bootstrap-based model training

After the data processing, the first main part of the framework aims to capture two types of uncertainties.

Model uncertainty is mainly caused by modeling inaccuracy. A neural network (NN) consists of model structure and model parameters, that is the performance of the NN is decided by these two elements. For the models with similar accuracy, the prediction uncertainties arise from their structures and parameters. Therefore, a series of models are constructed which vary from each other to capture model uncertainty. The model series is denoted as Φ , $\Phi = \{\phi_m\}_{m=1,2,\dots,M}$. Details of ϕ_m are listed in Table 1.

Table 1. Constructed model series

Model	Structure	Parameter	Data
ϕ_3	4 CNN	kernel: (10,1) channel:10	70% data
ϕ_6	5 CNN	kernel: (10,1) channel:10	70% data
ϕ_9	2 LSTM	hidden: 32/64 layer:1	70% data
ϕ_1	3 CNN	kernel: (10,1) channel:10	all data
ϕ_2	3 CNN	kernel: (10,1) channel:10	all data
ϕ_4	4 CNN	kernel: (10,1) channel:10	all data
ϕ_5	4 CNN	kernel: (10,1) channel:10	all data
ϕ_7	5 CNN	kernel: (10,1) channel:10	all data
ϕ_8	5 CNN	kernel: (10,1) channel:10	all data
ϕ_{10}	2 LSTM	hidden:32/64 layer:1	all data

Data uncertainty originates from measurement error, transmission noise, storage technology, etc. This kind of uncertainty ends up in the data set (X, Y) , $X, Y \in \mathbb{R}^{T \times N}$, where T denotes time window size, and N denotes the number of features. Bootstrap is a resampling technology and it selects different parts of data every time. Since the data uncertainty in sequential feature data could be changeable over time, bootstrap is able to extract data from different parts of the lifetime with heteroscedastic uncertainty. Groups of data sampled here are treated as training sets considering data uncertainty, which are represented by $(X, Y)^{(B)}$, $X^{(B)}, Y^{(B)} \in \mathbb{R}^{T \times N}$.

Model and data diversities are consequently added to the operation. Use the bootstrapped data $(X, Y)^{(B)}$ to train the models in Φ . After that, with the whole dataset X input into the trained models, every ϕ_m produce a new RUL list Y_i^* , where $Y_i^* = \{y_1^*, y_2^*, \dots, y_t^*\}$. M results are concatenated to obtain a prediction matrix Y , $Y \in \mathbb{R}^{t \times M}$, where t denotes the time length. In order to integrate the information of two types of uncertainties, the items in Y are fused into a sequence of new labels Y^* as follows:

$$Y^* = \frac{1}{M} \sum_{i=1}^M Y_i^* \quad (4)$$

where $Y^* \in \mathbb{R}^{t \times 1}$, and it is the foundation of the next step. The detail of bootstrap-based model training is depicted in Figure 4.

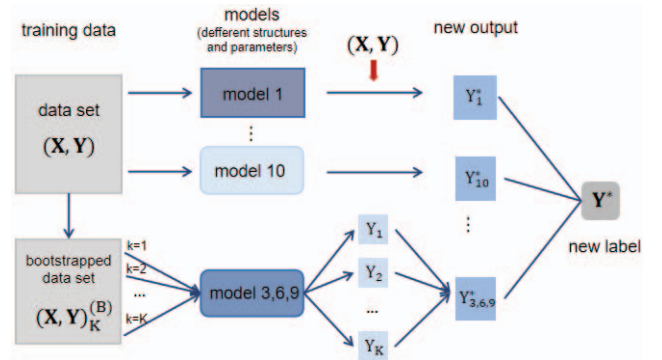


Figure 4. Bootstrap-based model training

Table 2. HNN for RUL prediction

Layer	Name	Parameter
1	input	$256 \times 30 \times 14$
2	CNN-tanh	kernel:(10,1) channel:10
3	CNN-tanh	kernel:(10,1) channel:10
4	CNN-tanh	kernel:(10,1) channel:1
5	LSTM	hidden:32 layer:1
6	LSTM	hidden:64 layer:1
7	Linear	neuron:8
8	Output1	neuron:1
	Output2 -sigmoid	neuron:1

2) HNN output

The second main part of the framework is designed to quantify uncertainties in RUL prediction and output the point estimation at the same time.

The heteroscedastic neural network can produce quantified uncertainty with convenience and efficiency. To extract features in degradation and retain time sequential information, a CNN-LSTM-based heteroscedastic neural network is constructed.

The result of each time step is assumed to be a normal distribution, so the output of the network has two neurons that represent RUL point estimation (mean) and the according uncertainty (heteroscedastic variance) respectively. New label Y^* contains model and data uncertainties, which is used to constitute a new training set $\mathcal{D}^*$ with the feature sequence matrix X , and NLL is chosen to be the loss function to fit the model. The base model is a 3-layer-CNN-2-layer-LSTM network as specifically shown in Table 2. In order to give a positive limitation to the variance, the sigmoid activation function is adopted.

3.2. Metrics

Point estimation and uncertainty quantification are both evaluated from the following two aspects of metrics:

1) Point estimation

Root mean square error (RMSE): RMSE measures the point prediction accuracy:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i^*)^2} \quad (5)$$

where $\hat{y}_i$ is the predictive value, y_i^* is label.

2) Uncertainty quantification

Prediction interval coverage probability (PICP): PICP describes the interval prediction accuracy.

$$PICP = \frac{1}{N} \sum_{i=1}^N C_i \quad (6)$$

where $C_i=1$ if $y_i^* \in [\hat{L}_i, \hat{U}_i]$, $C_i=0$ if $y_i^* \notin [\hat{L}_i, \hat{U}_i]$, $\hat{L}_i$ and $\hat{U}_i$ are estimated lower and upper boundaries.

Normalized mean prediction interval width (NMPIW): NMPIW evaluates the width of the prediction interval.

$$NMPIW = \frac{1}{RN} \sum_{i=1}^N (U_i - L_i) \quad (7)$$

where R denotes the range of the target and is used for the normalized purpose.

Coverage-width based criterion (CWC): CWC combines the above two metrics to provide a comprehensive indicator.

$$CWC = NMPIW (1 + \gamma e^{-\eta(PICP - \mu)}) \quad (8)$$

where $\gamma=0$ if $PICP \geq \mu$, $\gamma=1$ if $PICP \leq \mu$, and μ is decided by confidence interval level.

4. NUMERICAL EXPERIMENT

In this section, we use CMAPSS datasets to validate the effectiveness of our method in RUL point estimation and interval prediction. This dataset is generated from the simulation of the turbofan engine degradation which is provided by NASA Ames prognostics data repository. There are four subsets FD001, FD002, FD003, and FD004, the first one is the most used with one fault mode and one operating condition. FD001 is used in this paper.

4.1. Data Processing

Raw data are not suitable for neural networks, data pre-processing is necessary for high-quality prediction.

Feature selection: there are 21 sensor parameters in this dataset, however, some variables keep constant and do not change over time. Therefore, they are considered to provide no valid information and dropped. We finally choose 14 features as input and their indexes are 2, 3, 4, 7, 8, 9, 11, 12, 13, 14, 15, 17, 20, and 21.

Normalization: data with similar scales perform better in a neural network. Min-max normalization as follows is used to limit the data range into $[-1,1]$.

$$x_{norm} = 2 \times \frac{x_{max} - x}{x_{max} - x_{min}} - 1 \quad (9)$$

Time window processing: RUL prediction is a time sequential problem, sliding time window is able to contain more temporal information. We choose a window size of 30, which means 30 time step of high dimensional features, and the last one of the labels are grouped into one sample as (X, y) , where $X \in \mathbb{R}^{30 \times 14}$, $y \in \mathbb{R}^{1 \times 1}$. Finally, we get 17731 groups of training samples. The shape of the input data is (17731,30,14).

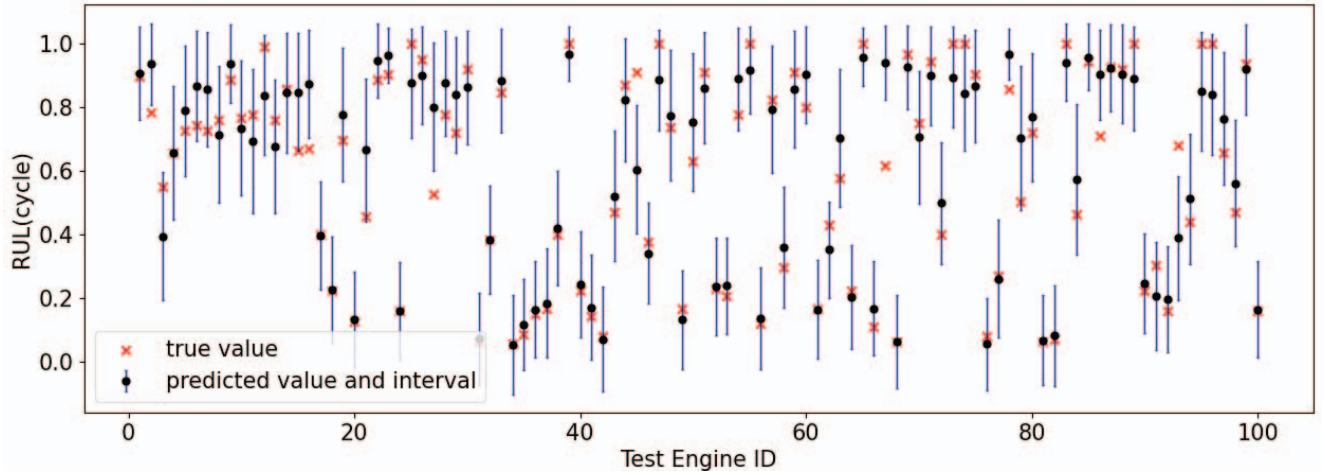


Figure 5. Prognostics results of Bootstrap-HNN model on test set FD001

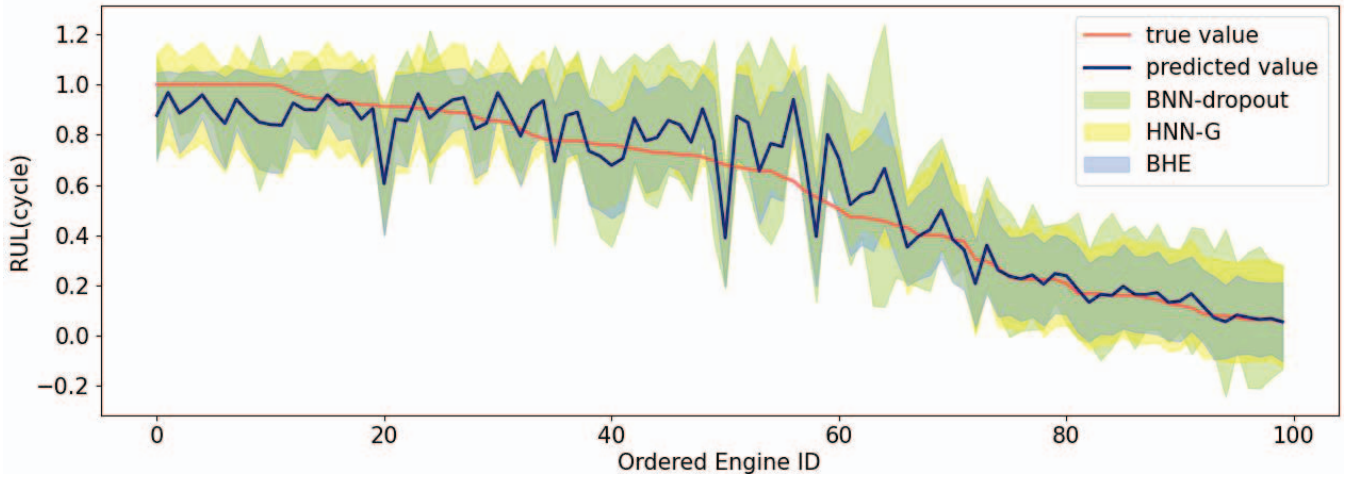


Figure 6. 95% prediction interval of RUL

Table3. Results of different methods

	BNN-dropout	General HNN	Bootstrap-HNN
PICP	0.87	0.92	0.92
NMPIW	0.42	0.40	0.33
CWC	1.35	0.94	0.77
RMSE	13.77	13.51	12.69

4.2. Result and Discussion

We apply the above trained Bootstrap-HNN model to test datasets of FD001 and get the last cycle value of every 100 engines. The result is shown in Figure 5. It can be found that most of the real points fall within the prediction interval while predicted points are close to real points. This proves our method's effectiveness in RUL point prediction and interval prediction.

Other two popular methods are used for comparison,

namely BNN-dropout and the general HNN model. It is worth noting that these three models maintain a consistent base structure, that is a combined CNN-LSTM model as shown in Table 2. According to the metrics introduced in 3.2, η and μ in CWC are set to 10 and 0.95 respectively. We can get the evaluation results in Table 3. Compared with the BNN-dropout method, general HNN has better performance both in point prediction and interval prediction. It also can be seen that, in the case of the same PICP, Bootstrap-HNN has narrower interval width, lower CWC, and better RMSE than general HNN.

In order to depict the comparison clearly, we sort the predicted value by true RUL label in descending order and plot the results of three methods in the same picture as shown in Figure 6. Apparently, all three intervals can cover the most of true values of these 100 tests while Bootstrap-HNN holds the narrowest width. Specifically, the width of the BNN-dropout result is narrower when the RUL value is high,

which indicates that model uncertainty is a major component of the source of uncertainty at the early stage of equipment degradation.

BNN considers model parameters as contributions instead of point values, general HNN measures uncertainty coming from feature data, and Bootstrap-HNN captures uncertainty both in models and data. Consequently, Our approach takes into account a more comprehensive picture and provides a better prediction.

5. CONCLUSION

This paper proposed a new framework to quantify uncertainty during RUL estimation through prediction interval. Bootstrap-HNN is a generic model for RUL prediction with data uncertainty and model uncertainty considering, which is meaningful for maintenance strategy making. Bootstrap-HNN consists of two parts, the first part contains a series of constructed models with different structures and parameters, and then trains them with bootstrapped samples. Another part is an HNN model based on the CNN-LSTM network with new labeled input generated from the first part. A numerical experiment on CMAPSS datasets validates the ability to capture the uncertainty of the proposed method, and it holds better performance on both prediction interval and point estimation according to evaluation metrics results.

However, due to the model property, two-times training is needed and it brings challenges to real-time RUL prediction. In the future, modeling simplification should be further investigated.

ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China (NSFC) under 61973269, and in part by the National Key Research and Development Program of China under 2019YFB1705502.

REFERENCE

- [1] Ferreira, Carlos, and Gil Gonçalves. "Remaining Useful Life prediction and challenges: A literature review on the use of Machine Learning Methods." *Journal of Manufacturing Systems* 63 (2022): 550-562.
- [2] Paris, Paul, and Fazil Erdogan. "A critical analysis of crack propagation laws." (1963): 528-533.
- [3] Wang, Youdao, Yifan Zhao, and Sri Addepalli. "Remaining useful life prediction using deep learning approaches: A review." *Procedia manufacturing* 49 (2020): 81-88.
- [4] Yang, Yun, et al. "A Battery Capacity and RUL Prognostic Approach Based on ARIMA and PF." *World Scientific Research Journal* 7.5 (2021): 220-232.
- [5] Wang, Han, et al. "Remaining useful life prediction and optimal maintenance time determination for a single unit using isotonic regression and gamma process

model." *Reliability Engineering & System Safety* 210 (2021): 107504.

[6] Yan, Bingxin, et al. "Two-stage physics-based Wiener process models for online RUL prediction in field vibration data." *Mechanical Systems and Signal Processing* 152 (2021): 107378.

[7] Lee, Myeong-Seok, Tanvir Alam Shifat, and Jang-Wook Hur. "Kalman Filter Assisted Deep Feature Learning for RUL Prediction of Hydraulic Gear Pump." *IEEE Sensors Journal* 22.11 (2022): 11088-11097.

[8] Wang, Yujie, et al. "A comprehensive review of battery modeling and state estimation approaches for advanced battery management systems." *Renewable and Sustainable Energy Reviews* 131 (2020): 110015.

[9] Sankararaman, Shankar. "Significance, interpretation, and quantification of uncertainty in prognostics and remaining useful life prediction." *Mechanical Systems and Signal Processing* 52 (2015): 228-247.

[10] Cai, Baoping, et al. "Remaining useful life re-prediction methodology based on Wiener process: Subsea Christmas tree system as a case study." *Computers & Industrial Engineering* 151 (2021): 106983.

[11] Yu, Wennian, et al. "A nonlinear-drift-driven Wiener process model for remaining useful life estimation considering three sources of variability." *Reliability Engineering & System Safety* 212 (2021): 107631.

[12] Song, Kai, and Lirong Cui. "A common random effect induced bivariate gamma degradation process with application to remaining useful life prediction." *Reliability Engineering & System Safety* 219 (2022): 108200.

[13] Li, Gaoyang, et al. "A Bayesian deep learning RUL framework integrating epistemic and aleatoric uncertainties." *IEEE Transactions on Industrial Electronics* 68.9 (2020): 8829-8841.

[14] Lin, Yan-Hui, and Gang-Hui Li. "A Bayesian deep learning framework for RUL prediction incorporating uncertainty quantification and calibration." *IEEE Transactions on Industrial Informatics* 18.10 (2022): 7274-7284.

[15] Kim, Minhee, and Kaibo Liu. "A Bayesian deep learning framework for interval estimation of remaining useful life in complex systems by incorporating general degradation characteristics." *IIEE Transactions* 53.3 (2020): 326-340.

[16] Blundell, Charles, et al. "Weight uncertainty in neural network." *International conference on machine learning*. PMLR, 2015.

[17] Gal, Yarin, and Zoubin Ghahramani. "Dropout as a bayesian approximation: Representing model uncertainty in deep learning." *international conference on machine learning*. PMLR, 2016.

[18] Peng, Weiwen, Zhi-Sheng Ye, and Nan Chen. "Bayesian deep-learning-based health prognostics toward prognostics uncertainty." *IEEE Transactions on Industrial Electronics* 67.3 (2019): 2283-2293.

[19] Huang, Dengshan, et al. "Bayesian neural network

based method of remaining useful life prediction and uncertainty quantification for aircraft engine." 2020 IEEE International Conference on Prognostics and Health Management (ICPHM). IEEE, 2020.

[20] Wang, Huanjie, Xiwei Bai, and Jie Tan. "Uncertainty quantification of bearing remaining useful life based on convolutional neural network." 2020 IEEE Symposium Series on Computational Intelligence (SSCI). IEEE, 2020.

[21] Nguyen, Khanh TP, Kamal Medjaher, and Christian Gogu. "Probabilistic deep learning methodology for uncertainty quantification of remaining useful lifetime of multi-component systems." Reliability Engineering & System Safety 222 (2022): 108383.

[22] Hesterberg, Tim. "Bootstrap." Wiley Interdisciplinary Reviews: Computational Statistics 3.6 (2011): 497-526.

[23] Schapire, Robert E. "A brief introduction to boosting." Ijcai. Vol. 99. 1999.

[24] Basora, Luis, et al. "A Benchmark on Uncertainty Quantification for Deep Learning Prognostics." arXiv preprint arXiv:2302.04730 (2023).